EEE3096S Practical 1A

STM32 GPIO, Timers, and Interrupts

HAZEL MPAKATI MPKHAZ001
TINASHE MASVORE MSVTIN003

August 14, 2026

1 Hardware Orientation

1. GPIO pins connected to the 8 user LEDs: PB0, PB1, PB2, PB3, PB4, PB5, PB6, and PB7 (GPIOB Pin 0 to Pin 7)
2. GPIO pins connected to the 4 push buttons: PA0, PA1, PA2, and PA3 (GPIOA Pin 0 to Pin 3
3. Pins connected to the 8 MHz crystal: PF0-OSC_IN and PF1-OSC_OUT.
4. Default system clock frequency using the internal HSI oscillator: 8 MHz
5. Purpose of the loopback header P2: To enable external signal routing, loopback verification, and diagnostic hardware testing between peripheral signals.

2 Task 1: Timer Configuration and Verification

2.1 Predictions

Record these before you write any code or calculate any values.

- System clock frequency: 8 MHz
- Expected prescaler value: 7999
- Expected ARR value: 999
- Expected interrupt period (s): 1.0 s

2.2 Calculations

Show your working for the timer clock frequency and the ARR value.

$$\text{Timer Clock} = \frac{\text{System Clock}}{\text{Prescaler} + 1} = \frac{8\,000\,000}{7999 + 1} = \frac{8\,000\,000}{8000} = 1000\,\text{Hz}$$

$$\text{ARR} = \frac{\text{Timer Clock}}{\text{Update Frequency}} - 1 = \frac{1000\,\text{Hz}}{1\,\text{Hz}} - 1 = 999$$



Figure 1: Oscillosope image of period an frequency

2.3 Measurement and Verification

- Measured period from the oscilloscope: 2.00 s total toggle period (1.00 s high / 1.00 s low)
- Comparison with your expected period: The measured interrupt period of 1.00 s matches the calculated prediction exactly. The observed pin output toggle period on PB0 is 2.00 s because the ISR toggles state once every 1.00 s update event.

3 Task 2: LED Control with Timer Interrupt

Confirm your running light video is in your submission folder, or link it here: Video in folder titled : TASK 2... **Video link:** UCT Cloud Task 2 Video (Running Light)

4 Task 3: Button Input and Software Debounce

4.1 Records

- Expected switch bounce duration:10 ms to 20 ms
- Maximum bounce duration measured on the oscilloscope: 10.9 ms
- Chosen software debounce delay time: 50 ms
- Formula to calculate the ARR value from the desired period in milliseconds:

$$\text{ARR} = T_{\text{ms}} - 1$$

4.2 Question 4 - VIDEO IN FOLDER TITLED: TASK 3 VIDEO

Video link: UCT Cloud Task 3 Video

When you update the ARR register dynamically, does the timer frequency change immediately mid-cycle, or does the timer wait for the next update event? Base your answer on the STM32 timer preload register behaviour.

In our initialization, `AutoReloadPreload` is set to `TIM_AUTORELOAD_PRELOAD_ENABLE` (`ARPE` bit set in `TIM16->CR1`). Under standard preload operation, writing a new value to `ARR` updates the preload register, but the active shadow register is only updated on the subsequent Update Event (UEV) when the counter reaches the old match value and overflows.



Figure 2: Oscilloscope Debounce Time Period Image

However, in our software implementation (`change_timer_period`), we explicitly write to `htim16.Instance->ARR = new_arr` and immediately reset the counter register by setting `htim16.Instance->CNT = 0`. Clearing `CNT` directly resets the counting sequence back to zero mid-cycle, causing the new period interval to take effect immediately rather than waiting for the previous cycle to expire.

=: Normally, because preload is enabled (`TIM_AUTORELOAD_PRELOAD_ENABLE`), updating ARR writes to a holding buffer. The live timer continues using its original limit until the current cycle finishes and triggers an Update Event.

However, in our `change_timer_period` function, we update ARR and immediately force `htim16.Instance->CNT = 0`. Resetting CNT directly aborts the current counting cycle, causing the new timer period to take effect immediately mid-cycle rather than waiting for the previous cycle to expire.

5 Task 4: Multi-Mode LED Control-Video in folder titled : TASK 4

Video link: UCT Cloud Task 4 Video

5.1 State Machine Explanation

Explain your state machine implementation for Mode 3. Describe the SPARKLE_IDLE, SPARKLE_DISPLAY, and SPARKLE_TURN_OFF states, and how your code moves between them. State Machine Explanation
Mode 3 (Sparkle Mode) is governed by a three-state non-blocking finite state machine driven inside `mode3_update()` using `HAL_GetTick()`:

1. `SPARKLE_IDLE`: Generates an 8-bit pseudo-random pattern using `rand() % 256`. It iterates through the bit pattern to illuminate the designated LEDs on GPIOB, records active LED indices in `sparkle_leds_on[]`, calculates a random hold time between 100 ms and 1500 ms, sets `sparkle_display_until = now + hold_duration`, and transitions to `SPARKLE_DISPLAY`.

2. `SPARKLE_DISPLAY`: Acts as a non-blocking delay state. It continuously evaluates system ticks until `now >= sparkle_display_until`. Upon expiration, it initializes the turn-off index `sparkle_off_index = 0`, sets the first random step delay between 100 ms and 150 ms, and transitions to `SPARKLE_TURN_OFF`.

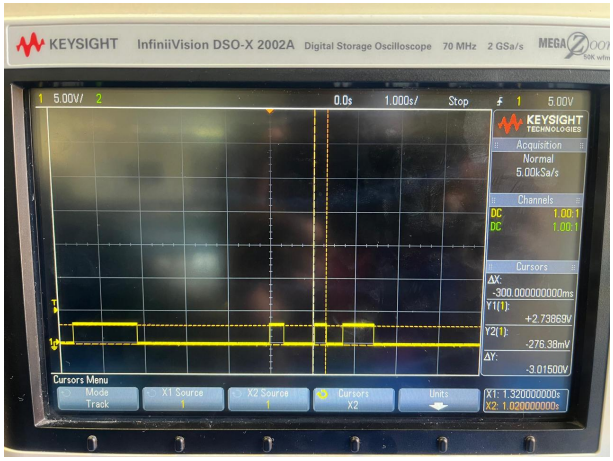


Figure 3: Task 4 Fastest Period



Figure 4: Task 4 Slowest Period

3. `SPARKLE_TURN_OFF`: Sequentially turns off active LEDs one by one at individual random intervals of 100 ms–150 ms. Once `sparkle_off_index` reaches `sparkle_num_leds_on` (all LEDs turned off), the state machine transitions back to `SPARKLE_IDLE` to initiate a new cycle.

5.2 Measurements

- Measured minimum LED-on time for Mode 3: 300 ms
- Measured maximum LED-on time for Mode 3: 1.5 s

6 Task 5: ADC Interrupts and LED Level Meter

6.1 Records

- Predicted 12-bit ADC register value, in decimal, for a 2.0 V input:

$$\text{ADC}_{\text{pred}} = \left\lfloor \frac{2.0\,\text{V}}{3.3\,\text{V}} \times 4095 \right\rfloor = 2482 \text{ (decimal)}$$

- Measured multimeter voltage at the moment the 5th LED turns on: 1.82 V



Figure 5: task 5 voltage image

6.2 Question

If a student accidentally sets the signal generator to output a 5.0 V DC signal into PA5, what physical damage might occur? Consult the I/O structure column for PA5 in the STM32F051x datasheet.

According to the STM32F051x datasheet, pin PA5 has an I/O structure designated as ***TC*** *(Standard* 3.3 *V I/O, non-5V tolerant). Unlike* ***FT*** *(*5 *V-tolerant) pins,* ***TC*** *pins lack the internal protective clamp circuit designed to handle input voltages above* $V_{DD} + 0.3$ *V (approx.* 3.6 *V).*

Applying a 5.0 *V signal to PA5 forward-biases the internal upper ESD protection diode connected between PA5 and the* V_{DD} *supply rail. This causes excessive current flow directly through the pin into the analog multiplexer and* V_{DD} *domain, leading to thermal overload, latch-up, and permanent destruction of PA5 and the internal ADC peripheral switch matrix.*

7 Task 6: Software PWM via Timer Interrupts

7.1 Records

– *Calculated Timer Prescaler and ARR values for exactly 10 kHz:*

$$Prescaler\ (PSC) = 7, \quad Auto\text{-}Reload\ (ARR) = 99$$

$$Frequency = \frac{8\,000\,000}{(7+1) \times (99+1)} = \frac{8\,000\,000}{800} = 10\,000\ Hz = 10\ kHz$$

– *Measured total period of the signal:* 9.994 *ms (*100.06 *Hz signal frequency)*

– *Measured positive pulse width:* 3.00 *ms (*30.00% *duty cycle)*

7.2 Question

A project requires a 50 Hz signal with a duty cycle resolution of 1 percent. What is the exact timer interrupt frequency required? Show your calculation.



Figure 6: Positive Half Cycle Period

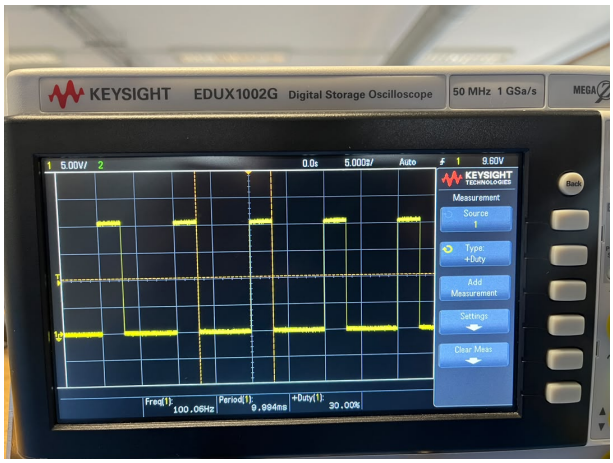


Figure 7: Full Cycle Period and Duty cycle

To achieve a 1% *duty cycle resolution, the software counter must divide one period of the output signal into 100 equal discrete time steps (*$100\%/1\% = 100$*).*

The required timer interrupt frequency is $5000\,\boldsymbol{Hz}$ *(or* $5\,\boldsymbol{kHz}$*).*